

Oracle → BigQuery

dbt Migration

Agent Knowledge Base

Medallion Architecture · CrewAI Pipeline · PL/SQL Conversion Reference



Purpose	Reference knowledge for CrewAI agents executing Oracle-to-BigQuery migration
Architecture	Medallion (Bronze / Silver / Gold)
Target Platform	Google BigQuery + dbt Core
Source Platform	Oracle Database (PL/SQL)
Agent Framework	CrewAI multi-agent pipeline
Document Scope	Discovery · Classification · Schema Conversion · Code Generation · Validation · Supervision

1. Pipeline Overview

The migration is executed as a sequential six-phase CrewAI pipeline. Each phase is owned by a dedicated specialist agent. The Supervisor Agent sits above all phases, enforces quality gates between them, and is the sole agent authorised to delegate work. No agent may begin work until its upstream phase has cleared its quality gate.

Phase	Agent	Task	Key Output
0 — Pre-flight	supervisor_agent	supervise_migration_task	Connectivity verified, config loaded
1 — Discovery	discovery_agent	discover_oracle_schema_task	Full object inventory + medallion tags
2 — Classification	classifier_agent	classify_complexity_task	Complexity tiers + topological sort
3 — Schema Conversion	schema_conversion_agent	convert_schemas_task	BQ tables + dbt model stubs (bronze/silver/
4 — Code Generation	codegen_agent	convert_low_medium_procs_task	dbt model SQL + schema.yml per layer
5 — Validation	validator_agent	validate_migrations_task	PASS/FAIL per object + lineage check
6 — Sign-off	supervisor_agent	supervise_migration_task	Final Migration Completion Report

Pipeline invariant: BRONZE objects are always processed before SILVER, and SILVER before GOLD. The supervisor enforces this ordering via topological sort output from the classifier agent. No exceptions.

2. Medallion Architecture

The migration follows the medallion architecture pattern with three layers. Every Oracle object is assigned to exactly one layer. The layer determines the dbt folder, materialisation strategy, and reference pattern used.

BRONZE Raw / Landing	SILVER Transformation	GOLD Reporting / KPI
Oracle tables and views whose name starts with SRC or src . Declared in <code>models/bronze/sources.yml</code> as dbt sources. Referenced downstream using <code>source()</code> . BigQuery table is physically created via DDL. dbt model: thin <code>SELECT</code> wrapper over the source.	Oracle tables, views, and procedures that transform or enrich Bronze data. Placed under <code>models/silver/</code> . Reference Bronze via <code>ref()</code> — NEVER <code>source()</code> . Materialised as view or table. LOW procs: <code>materialized='table'</code> ; MEDIUM: <code>incremental</code> .	Oracle tables/views/procedures that aggregate Silver data for reporting. Placed under <code>models/gold/</code> . Reference Silver (and Bronze) via <code>ref()</code> exclusively. Materialised as table. Consumer-facing: KPIs, dashboards, final fact/dim tables.

dbt Project Folder Structure

```
dbt_project/  
models/  
  bronze/  
    sources.yml # SRC/src source declarations ONLY  
    src_customers.sql # thin source() wrapper  
    src_orders.sql  
  silver/  
    schema.yml # column descriptions + dbt tests  
    orders_enriched.sql # ref('src_orders') – uses ref()  
    customer_metrics.sql # ref('src_customers')  
  gold/  
    schema.yml  
    revenue_summary.sql # ref('orders_enriched') – uses ref()  
  macros/ # (utility Jinja macros only – no procedures)
```

Hard rule — source() vs ref(): source() is permitted ONLY inside models/bronze/ files. Every Silver and Gold model must use ref() for all upstream references. Any violation is treated as a blocking issue by the validator and supervisor agents.

3. Agent Roles & Responsibilities

supervisor_agent

Migration Project Supervisor & Quality Gate Enforcer

- Orchestrates all five specialist agents in strict pipeline order.
- Performs pre-flight connectivity checks (Oracle + BigQuery) before delegating.
- Enforces quality gates after every phase — halts and escalates on any failure.
- Maintains and publishes the human_review_queue after each phase.
- Produces the Final Migration Completion Report with 7 structured sections.
- The ONLY agent with allow_delegation: true. All others are false.
- Tools: read_migration_log, write_migration_log, read_config, list_tables (1 call), count_rows_bq (1 call).
- max_iter: 50.

discovery_agent

Oracle Database Discovery & Medallion Layer Analyst

- Calls list_tables, list_views, list_procedures to enumerate all Oracle objects.
- Calls get_table_ddl or get_view_ddl for every table and view.
- Calls get_proc_body + get_dependencies for every procedure and function.
- Tags every object with medallion_layer: BRONZE (SRC/src prefix) or SILVER_CANDIDATE.
- Writes every object to the migration log with status=DISCOVERED.
- Never guesses — if an object is absent it reports it as absent.
- max_iter: 20.

classifier_agent

Migration Complexity Classifier & Medallion Layer Strategist

- Reads the full discovery output and applies the LOW / MEDIUM / HIGH scoring rubric.
- Assigns final medallion_layer (BRONZE / SILVER / GOLD) to every object.
- Assigns migration_strategy (dbt_model / human_review) per object.
- Produces a topologically sorted migration order: BRONZE first, then SILVER, then GOLD.
- Writes HIGH complexity objects to the human_review_queue.
- Conservative scorer — when in doubt, scores higher not lower.
- max_iter: 10.

schema_conversion_agent

Oracle-to-BigQuery DDL Schema Converter & dbt Project Architect

- Creates BigQuery DDL tables for Bronze (SRC/src) objects ONLY.
- Generates models/bronze/sources.yml with SRC/src entries only — no non-SRC tables.
- Generates .sql model files under models/bronze/, models/silver/, models/gold/.
- Bronze models use source(); Silver and Gold models use ref() exclusively.
- Applies Oracle → BigQuery type mapping rules from schema_map.json.
- Applies partitioning (>1M rows) and clustering (from WHERE clause analysis).
- max_iter: 20.

codegen_agent

PL/SQL to dbt BigQuery Code Generation Engineer

- Converts all LOW and MEDIUM complexity Oracle procedures/functions to dbt models.
 - ALL procedures become dbt model .sql files — no dbt macros are created for procedures.
 - LOW procedures: materialized='table'. MEDIUM procedures: materialized='incremental'.
 - Places models in models/silver/ or models/gold/ based on medallion_layer classification.
 - All inter-model references use ref(); never raw dataset.table strings.
 - Generates/appends schema.yml per layer with column descriptions and dbt tests.
 - Processes in batches of 10; writes progress to migration log after each batch.
 - max_iter: 30.
-

validator_agent

Validation, Data Reconciliation & dbt Lineage Integrity Engineer

- Runs 3 ordered checks per object: SQL syntax dry-run, row count reconciliation, column sample.
 - Stops at first failure — does not proceed to subsequent checks on a failing object.
 - Row count tolerance: 0.01% ($\text{diff_pct} = \text{abs}(\text{oracle} - \text{bq}) / \text{oracle} * 100$).
 - Column sample: 50 rows; fails if >2% of sampled rows mismatch on any non-nullable column.
 - Enforces lineage: no source() in Silver/Gold, no raw table strings, all ref() calls resolve.
 - Flags DDL_IN_MODEL if any procedure-originated model contains CREATE/INSERT statements.
 - On 3 consecutive FAILs: escalates object to HUMAN_REVIEW status.
 - max_iter: 20.
-

4. Complexity Scoring Rubric

The classifier_agent applies this rubric to every Oracle procedure and function. Scores are deterministic — the same input always produces the same score. When two criteria conflict, the higher score always wins.

Criterion	LOW	MEDIUM	HIGH
SQL patterns	Pure SELECT/INSERT/UPDATE, aggregations, window fns	Single CURSOR FOR loop, MERGE/UPSERT patterns	Nested cursors, EXECUTE IMMEDIATE, dynamic SQL of any kind
Procedure calls	None	Calls 1–2 LOW procedures	Calls HIGH procedures or DBMS_* packages
Exception handling	None or trivial	WHEN OTHERS basic	Complex nested handlers
Dynamic SQL	None	None	Any EXECUTE IMMEDIATE or DBMS_SQL
Cross-schema deps	None	None	Any (owner != schema)
DB links	None	None	Any @REMOTE_DB reference
Dependency count	0 – 3 objects	3 – 8 objects	> 8 objects
Automation confidence	>= 0.90	0.75 – 0.89	< 0.75
Migration strategy	dbt model (table)	dbt model (incremental)	HUMAN_REVIEW queue

Conservative scoring rule: classifier_agent scores higher when uncertain. A MEDIUM that is actually HIGH wastes one human review cycle. A HIGH scored as LOW can corrupt the target dataset. Err upward.

5. Oracle → BigQuery Type Mapping

The `schema_conversion_agent` applies these mappings when converting Oracle DDL to BigQuery CREATE TABLE statements. The agent always consults `schema_map.json` first; this table is the canonical fallback reference.

Oracle Type	BigQuery Type	Notes / Warnings
NUMBER(p, s)	NUMERIC(p, s)	Exact numeric — preserve precision and scale.
NUMBER	FLOAT64	WARNING: precision loss possible. Flag in report.
VARCHAR2(n)	STRING	Encoding handled by BigQuery; length hint dropped.
CHAR(n)	STRING	Oracle CHAR pads with spaces; BigQuery STRING does not.
DATE	DATETIME	Oracle DATE always contains time; maps to DATETIME not DATE.
TIMESTAMP	TIMESTAMP	Direct mapping. Timezone info preserved if present.
CLOB	STRING	10 MB practical limit in BigQuery. Flag large CLOBs.
BLOB	BYTES	Use only for truly binary data. Verify downstream handling.
INTEGER	INT64	Direct mapping.
BOOLEAN	BOOL	Direct mapping.
INTERVAL YEAR TO MONTH	INT64 (months)	Store as integer months; document in schema.yml.
XMLTYPE	STRING	Store as raw XML string; document for downstream parsing.
RAW(n)	BYTES	Binary data — verify consumer compatibility.

Partitioning & Clustering Rules

Partitioning trigger	Tables with > 1,000,000 rows receive partitioning.
Partition column	First DATE or TIMESTAMP column found in the schema. If none, use <code>_PARTITIONTIME</code> pseudo-column.
Clustering trigger	Columns appearing in WHERE clauses of procedures that reference this table (from dependency analysis).

Max clustering cols	4 columns — hard BigQuery limit. Prioritise by selectivity.
No partition needed	Tables with <= 1M rows. No partition clause generated.

6. PL/SQL → BigQuery SQL Conversion Patterns

The `codegen_agent` applies these conversions when translating Oracle PL/SQL procedure bodies into BigQuery standard SQL dbt models. Every conversion decision must be documented in an inline SQL comment at the point of change.

Oracle PL/SQL Pattern	BigQuery / dbt Equivalent
<code>CURSOR FOR loop</code>	CTE or subquery
<code>MERGE INTO ... USING</code>	BigQuery <code>MERGE INTO ... USING</code>
<code>NVL(a, b)</code>	<code>IFNULL(a, b)</code>
<code>NVL2(a, b, c)</code>	<code>IF(a IS NOT NULL, b, c)</code>
<code>DECODE(x, v1, r1, ...)</code>	<code>CASE WHEN x=v1 THEN r1 ... END</code>
<code>SYSDATE</code>	<code>CURRENT_DATETIME()</code>
<code>SEQUENCE.NEXTVAL</code>	<code>GENERATE_UUID()</code> or <code>FARM_FINGERPRINT(...)</code>
<code>ROWNUM</code>	<code>ROW_NUMBER() OVER (ORDER BY ...)</code>
<code>a b (concat)</code>	<code>CONCAT(a, b)</code>
<code>TO_DATE(str, fmt)</code>	<code>PARSE_DATETIME(fmt, str)</code>
<code>TO_CHAR(date, fmt)</code>	<code>FORMAT_DATETIME(fmt, datetime_col)</code>
<code>TRUNC(date)</code>	<code>DATE(datetime_col)</code>
<code>ADD_MONTHS(d, n)</code>	<code>DATE_ADD(d, INTERVAL n MONTH)</code>
<code>LAST_DAY(d)</code>	<code>DATE_SUB(DATE_TRUNC(DATE_ADD(d, INTERVAL 1 MONTH), MONTH), INTERVAL 1 DAY)</code>
<code>SUBSTR(s, pos, len)</code>	<code>SUBSTR(s, pos, len) -- same syntax</code>
<code>INSTR(s, sub)</code>	<code>STRPOS(s, sub)</code>
<code>MONTHS_BETWEEN(d1, d2)</code>	<code>DATE_DIFF(d1, d2, MONTH)</code>
<code>IN/OUT parameters</code>	Resolved as CTEs or SELECT expressions in model body
<code>PROCEDURE CALL chain</code>	<code>ref()</code> to the upstream converted dbt model
<code>RAISE_APPLICATION_ERROR</code>	Remove; add dbt test or WARNING comment
<code>DBMS_OUTPUT.PUT_LINE</code>	Remove; log via dbt's <code>run_query</code> if needed

Procedure Model Template

Every converted procedure becomes a dbt model with this structure:

```
{{
config(
materialized = 'table', -- LOW complexity
-- OR --
materialized = 'incremental', -- MEDIUM complexity
unique_key = 'primary_key_column',
tags = ['silver'] -- or ['gold']
)
}}

-- Converted from Oracle procedure: PROC_NAME
-- Original location: schema.PROC_NAME
-- Conversion notes: CURSOR FOR loop at line 12 converted to CTE

with source_data as (
select *
from {{ ref('src_orders') }} -- Bronze reference via ref()
),

transformed as (
select
order_id,
customer_id,
IFNULL(discount_pct, 0) as discount_pct, -- NVL -> IFNULL
CURRENT_DATETIME() as processed_at, -- SYSDATE -> CURRENT_DATETIME()
ROW_NUMBER() OVER (PARTITION BY customer_id ORDER BY order_date)
as order_rank -- ROWNUM -> ROW_NUMBER()
from source_data
)

select * from transformed
```

7. sources.yml Structure (Bronze Layer Only)

The sources.yml file is written once by schema_conversion_agent and lives at models/bronze/sources.yml. It declares all SRC/src raw tables as dbt sources. Non-SRC tables and views must NEVER appear in this file.

```
version: 2

sources:
  - name: my_bq_dataset # logical source name = dataset name
    database: my_gcp_project
    schema: my_bq_dataset
    description: 'Raw Bronze layer - Oracle migration landing tables'
    tags: ['bronze', 'raw', 'oracle_migration']
    tables:
      - name: src_customers
        description: 'Raw customer records landed from Oracle SRC_CUSTOMERS'
        columns:
          - name: customer_id
            description: 'Primary key - Oracle NUMBER(10) mapped to INT64'
        tests:
          - not_null
          - unique
          - name: customer_name
            description: 'VARCHAR2(200) mapped to STRING'
          - name: created_date
            description: 'Oracle DATE (includes time) mapped to DATETIME'
        meta:
          partition_column: created_date # only for tables > 1M rows
          clustering_columns: ['customer_id', 'region_code']
```

sources.yml inclusion rule: A table qualifies for sources.yml if and only if its Oracle name starts with 'SRC' or 'src' (case-insensitive prefix match). All other tables and views are dbt models and are referenced via ref(), never source().

schema.yml Structure (Silver & Gold Layers)

Each of models/silver/schema.yml and models/gold/schema.yml is generated and appended to incrementally as codegen_agent converts each procedure.

```
version: 2

models:
  - name: orders_enriched
    description: >
      Converted from Oracle procedure PROC_ENRICH_ORDERS.
      Joins src_orders with src_customers to produce enriched order records.
    config:
      tags: ['silver']
    columns:
      - name: order_id
        description: 'Primary key. Oracle NUMBER(10) -> INT64.'
    tests:
      - not_null
      - unique
      - name: customer_id
        description: 'Foreign key to src_customers.'
        tests:
          - not_null
          - relationships:
              to: ref('src_customers')
              field: customer_id
      - name: discount_pct
        description: 'Discount percentage. Nullable - NVL defaulted to 0.'
```

8. Validation Protocol

The `validator_agent` runs four checks on every generated object in dependency order. A PASS requires all four checks to succeed. Failure in any check terminates validation for that object and triggers a retry or escalation.

CHECK 1	SQL Syntax Dry-Run Call <code>dry_run_bq</code> with the full .sql model content. BigQuery parses the SQL without executing it. Any parse error returns <code>SYNTAX_ERROR</code> with the exact BigQuery error message. Applies to all model types including those converted from procedures.
CHECK 2	Row Count Reconciliation Call <code>count_rows_oracle</code> on the source table or procedure output, then <code>count_rows_bq</code> on the materialised BigQuery table. Calculate: $\text{diff_pct} = \frac{\text{abs}(\text{oracle} - \text{bq})}{\text{oracle}} * 100$. Tolerance: 0.01%. Any <code>diff_pct</code> above this threshold returns <code>COUNT_MISMATCH</code> with both counts and the percentage difference.
CHECK 3	Column-Level Sample Validation Call <code>sample_rows_oracle(n=50)</code> and <code>execute_bq</code> on the same 50 primary keys. Compare all non-nullable columns. Fails if more than 2% of sampled rows mismatch on any column. Returns <code>DATA_MISMATCH</code> with column name, oracle value, and bq value.
CHECK 4	dbt Lineage Integrity Scan generated SQL: (a) no raw dataset.table strings; (b) <code>source()</code> only in Bronze models; (c) every <code>ref()</code> call resolves to an existing model file; (d) no DDL statements (<code>CREATE/INSERT/etc.</code>) in procedure-originated models. Failures: <code>INVALID_REF</code> or <code>DDL_IN_MODEL</code> .

Retry & Escalation Logic

retry_count < 3	Set status= <code>FAILED_VALIDATION</code> . <code>codegen_agent</code> retries with the exact failure reason as a corrective prompt.
retry_count >= 3	Set status= <code>HUMAN_REVIEW</code> . Object added to <code>human_review_queue</code> with full failure history.
PASS criteria	All 4 checks pass. Set confidence=1.0, status= <code>VALIDATED</code> . Append to <code>validation_report.json</code> .
Pass rate gate	Supervisor enforces minimum 95% validation pass rate before sign-off. Below 95% = blocking issue.
Processing order	Dependency-sorted — same order as code generation. Object is not validated until all its dependencies have status= <code>VALIDATED</code> .

9. Supervisor Quality Gates

The `supervisor_agent` enforces a quality gate after every phase. If any check fails, the pipeline halts immediately. The supervisor writes a `blocking_issue` entry to the migration log and surfaces an escalation report to the human operator. Work resumes only after the operator confirms the issue is resolved.

Gate 1 — Post Discovery

- Every object has `status=DISCOVERED` and a non-null `medallion_layer`.
- No object has a `NULL dependency_list` (must be `[]` if no dependencies).
- All `SRC/src`-prefixed objects are tagged `BRONZE`; all others `SILVER_CANDIDATE`.

Gate 2 — Post Classification

- Every object has complexity (`LOW/MEDIUM/HIGH`), `medallion_layer`, and `migration_strategy` set.
- Topological sort is valid: no object appears before any of its dependencies.
- All `HIGH` complexity objects are written to the `human_review_queue`.

Gate 3 — Post Schema Conversion

- `models/bronze/sources.yml` exists and contains ONLY `SRC/src` table entries.
- `models/bronze/`, `models/silver/`, `models/gold/` directories exist with `>= 1` `.sql` file each.
- Zero Silver or Gold `.sql` files contain a `source()` call.
- Zero `SCHEMA_FAILED` objects are unaddressed.

Gate 4 — Post Code Generation

- Every `LOW/MEDIUM` object has `status=GENERATED`.
- Every `dbt_model` object has a `.sql` file in `models/silver/` or `models/gold/`.
- Zero generated model files contain raw `dataset.table` strings.
- `schema.yml` entries exist for every generated model.

Gate 5 — Post Validation

- Every generated object has `status=VALIDATED` or `status=HUMAN_REVIEW`.
- No object is left in `status=GENERATED` or `FAILED_VALIDATION` without `retry_count >= 3`.
- Overall validation pass rate `>= 95%`.
- All Silver/Gold models confirmed `ref()`-only (no `source()`, no raw strings).

Blocking issue record format: `phase | failed_check | affected_objects (list) | recommended_action`.
Written to migration log with `status=BLOCKED`. Pipeline does not advance until the operator explicitly resolves or overrides the issue.

10. Migration Log Schema

Every agent reads and writes to the shared migration log via `write_migration_log` and `read_migration_log` tools. The log is the single source of truth for pipeline state. The following fields are written by each phase.

Field	Written By	Values / Format
<code>object_name</code>	<code>discovery_agent</code>	Oracle object name (uppercase)
<code>object_type</code>	<code>discovery_agent</code>	TABLE VIEW PROCEDURE FUNCTION
<code>schema</code>	<code>discovery_agent</code>	{oracle_schema}
<code>row_count</code>	<code>discovery_agent</code>	Integer (tables only); null for procs/views
<code>dependency_list</code>	<code>discovery_agent</code>	List of Oracle object names; [] if none
<code>raw_source</code>	<code>discovery_agent</code>	Full DDL or PL/SQL body string
<code>medallion_layer</code>	<code>discovery_agent</code> / <code>classifier_agent</code>	BRONZE SILVER GOLD SILVER_CANDIDATE
<code>status</code>	All agents	DISCOVERED CLASSIFIED SCHEMA_CONVERTED SCHEMA_FAILED
<code>complexity</code>	<code>classifier_agent</code>	LOW MEDIUM HIGH
<code>migration_strategy</code>	<code>classifier_agent</code>	dbt_model human_review
<code>migration_order_index</code>	<code>classifier_agent</code>	Integer (topological sort position)
<code>depends_on</code>	<code>classifier_agent</code>	List of object names this object depends on
<code>blocked_until</code>	<code>classifier_agent</code>	List of object names that must be VALIDATED first
<code>artifact_type</code>	<code>codegen_agent</code>	MODEL
<code>artifact_path</code>	<code>codegen_agent</code>	Relative path e.g. models/silver/orders.sql
<code>materialization</code>	<code>codegen_agent</code>	table incremental
<code>validation_result</code>	<code>validator_agent</code>	PASS FAIL
<code>confidence</code>	<code>validator_agent</code>	1.0 (PASS) 0.0 (FAIL)
<code>retry_count</code>	<code>validator_agent</code>	Integer 0–3
<code>failure_reason</code>	<code>validator_agent</code>	SYNTAX_ERROR COUNT_MISMATCH DATA_MISMATCH INVALID_SCHEMA
<code>blocking_issue</code>	<code>supervisor_agent</code>	Struct: phase, failed_check, affected_objects, recommended_action

11. Quick Reference Card

Layer Assignment Rules

BRONZE	Oracle object name starts with 'SRC' or 'src' (case-insensitive prefix). Tables AND views qualify. Physical BigQuery table created via DDL.
SILVER	All other tables/views that read from Bronze. Oracle views joining SRC tables. Procedures reading from SRC/src objects. dbt model under models/silver/.
GOLD	Tables/views/procedures aggregating Silver data for reporting or KPIs. dbt model under models/gold/.
Default	If layer cannot be determined from dependencies alone, default to SILVER and flag in migration log.

ref() vs source() Decision Rule

Use source()	ONLY in models/bronze/ files. Pattern: {{ source('dataset_name', 'table_name') }}
Use ref()	In ALL Silver and Gold models. Pattern: {{ ref('model_name') }}. Covers Bronze models, Silver models, and inter-procedure references.
Never use	Raw dataset.table strings inside any dbt model file.
Validation failure	Any source() in Silver/Gold = INVALID_REF. Any raw string = INVALID_REF. Both are blocking issues.

Procedure Conversion Decision

ALL procedures	Convert to dbt MODEL (.sql file in models/silver/ or models/gold/). No procedures become dbt macros.
LOW complexity	materialized='table', tags=['silver'] or ['gold'].
MEDIUM complexity	materialized='incremental', unique_key required, tags=['silver'] or ['gold'].
HIGH complexity	Do NOT convert. Add to human_review_queue with reason.
Procedure call chain	Calling procedure becomes dbt model; referenced procedure resolved via ref('called_proc_model_name').
IN/OUT parameters	Resolved as CTEs or intermediate SELECT expressions within the model body.

Supervisor Tools (Only These 5)

read_migration_log	Read phase status, object counts, blocking issues.
write_migration_log	Write phase summaries, blocking issues, pre-flight, final sign-off.
read_config	Pre-flight config validation.

list_tables	ONE call only — pre-flight Oracle connectivity check.
count_rows_bq	ONE call only — pre-flight BigQuery connectivity check.

Failure Type Reference

SYNTAX_ERROR	BigQuery dry-run rejected the generated SQL. Fix: correct SQL syntax in the model file.
COUNT_MISMATCH	Row count difference > 0.01% between Oracle source and BigQuery target.
DATA_MISMATCH	More than 2% of sampled rows differ in at least one non-nullable column.
INVALID_REF	source() in Silver/Gold model, or ref() pointing to a non-existent model file, or raw table string found.
DDL_IN_MODEL	Procedure-originated model contains CREATE TABLE, CREATE VIEW, INSERT, or other DDL/DML statements.
ORACLE_UNREACHABLE	Pre-flight: Oracle connectivity check failed.
BIGQUERY_UNREACHABLE	Pre-flight: BigQuery connectivity check failed.

12. Agent Configuration Summary

Agent	allow_delegation	max_iter	verbose	Key Constraint
supervisor_agent	TRUE	50	true	Only agent that can delegate. Enforces all quality
discovery_agent	false	20	true	Never guesses. Always calls tools. Tags every o
classifier_agent	false	10	true	Conservative scorer. Topological sort enforced.
schema_conversion_agent	false	20	true	sources.yml: SRC/src only. DDL for Bronze only.
codegen_agent	false	30	true	All procs → models. No macros from procedures
validator_agent	false	20	true	4 checks in order. 95% pass rate. Lineage integr

This document is generated from the authoritative agents.yaml and tasks.yaml configuration files. In case of any discrepancy, the YAML files take precedence. Update this document whenever the YAML files are modified.